```cpp
Node* minValueNode(Node* node) {
    Node* current = node;

    // Loop down to find the leftmost leaf (smallest value)
    while (current && current->left != nullptr) {
        current = current->left;
    }

    return current;
}

Node* deleteNode(Node* root, int key) {
    // Base case: if the tree is empty
    if (root == nullptr) return root;

    // Recursive cases
    if (key < root->data) {
        root->left = deleteNode(root->left, key);
    }
    else if (key > root->data) {
        root->right = deleteNode(root->right, key);
    }
    else {
        // Node with only one child or no child
        if (root->left == nullptr) {
            Node* temp = root->right;
            delete root;
            return temp;
        }
        else if (root->right == nullptr) {
            Node* temp = root->left;
            delete root;
            return temp;
        }

        // Node with two children: Get the inorder successor (smallest in the right subtree)
        Node* temp = minValueNode(root->right);

        // Copy the inorder successor's content to this node
        root->data = temp->data;

        // Delete the inorder successor
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}
```

```cpp
// Helper function to perform in-order traversal and store nodes in a vector
void storeInOrder(Node* root, std::vector<Node*>& nodes) {
    if (root == nullptr) return;
    storeInOrder(root->left, nodes);
    nodes.push_back(root);
    storeInOrder(root->right, nodes);
}

// Helper function to build a balanced BST from a sorted vector of nodes
Node* buildBalancedTree(std::vector<Node*>& nodes, int start, int end) {
    if (start > end) return nullptr;

    // Choose middle node to keep balance
    int mid = (start + end) / 2;
    Node* root = nodes[mid];

    // Recursively build the left and right subtrees
    root->left = buildBalancedTree(nodes, start, mid - 1);
    root->right = buildBalancedTree(nodes, mid + 1, end);

    return root;
}

// Function to balance the BST
Node* balanceBST(Node* root) {
    // Store nodes of BST in sorted order
    std::vector<Node*> nodes;
    storeInOrder(root, nodes);

    // Build balanced BST from sorted nodes
    return buildBalancedTree(nodes, 0, nodes.size() - 1);
}

// Utility function to print in-order traversal of the tree
void inOrderTraversal(Node* root) {
    if (root == nullptr) return;
    inOrderTraversal(root->left);
    std::cout << root->data << " ";
    inOrderTraversal(root->right);
}
```

```cpp
int height(Node* root) {
    if (root == nullptr) {
        return -1; // Return -1 for null nodes (height of an empty tree is -1)
    }

    // Calculate height of left and right subtrees
    int leftHeight = height(root->left);
    int rightHeight = height(root->right);

    // Height of the tree is the max height of left/right subtree + 1 (for the root)
    return std::max(leftHeight, rightHeight) + 1;
}
```

```cpp
int depth(Node* root, Node* target, int currentDepth) {
    // Base case: If root is null, return -1 indicating that the node was not found
    if (root == nullptr) {
        return -1;
    }

    // If the target node is found, return the current depth
    if (root == target) {
        return currentDepth;
    }

    // Check left and right subtrees for the target node
    int leftDepth = depth(root->left, target, currentDepth + 1);
    if (leftDepth != -1) {
        return leftDepth; // If found in left subtree, return its depth
    }

    return depth(root->right, target, currentDepth + 1); // Otherwise, check right subtree
}
```

```cpp
int evaluatePostfix(const std::string& expression) {
    std::stack<int> stack;
    std::istringstream tokens(expression); // Stream to read tokens from the expression
    std::string token;

    while (tokens >> token) { // Read each token
        if (isdigit(token[0])) { // Check if the token is an operand
            stack.push(std::stoi(token)); // Convert to integer and push onto stack
        }
        else { // The token is an operator
            int rightOperand = stack.top(); // Pop the top operand
            stack.pop();
            int leftOperand = stack.top(); // Pop the next operand
            stack.pop();

            // Apply the operator
            switch (token[0]) {
            case '+':
                stack.push(leftOperand + rightOperand);
                break;
            case '-':
                stack.push(leftOperand - rightOperand);
                break;
            case '*':
                stack.push(leftOperand * rightOperand);
                break;
            case '/':
                stack.push(leftOperand / rightOperand);
                break;
            default:
                std::cerr << "Unknown operator: " << token << std::endl;
                return -1;
            }
        }
    }

    return stack.top(); // The final result will be on top of the stack
}
```

```cpp
int evaluatePrefix(const std::string& expression) {
    std::stack<int> stack;
    std::istringstream tokens(expression); // Stream to read tokens from the expression
    std::string token;
    std::vector<std::string> tokenList;

    // First, store the tokens in a vector
    while (tokens >> token) {
        tokenList.push_back(token);
    }

    // Traverse the tokens in reverse order
    for (int i = tokenList.size() - 1; i >= 0; --i) {
        token = tokenList[i];
        if (isdigit(token[0])) { // Check if the token is an operand
            stack.push(std::stoi(token)); // Convert to integer and push onto stack
        }
        else { // The token is an operator
            int leftOperand = stack.top(); // Pop the first operand
            stack.pop();
            int rightOperand = stack.top(); // Pop the second operand
            stack.pop();

            // Apply the operator
            switch (token[0]) {
            case '+':
                stack.push(leftOperand + rightOperand);
                break;
            case '-':
                stack.push(leftOperand - rightOperand);
                break;
            case '*':
                stack.push(leftOperand * rightOperand);
                break;
            case '/':
                stack.push(leftOperand / rightOperand);
                break;
            default:
                std::cerr << "Unknown operator: " << token << std::endl;
                return -1;
            }
        }
    }

    return stack.top(); // The final result will be on top of the stack
}
```

```cpp
int countNodes(Node* root) {
    if (root == nullptr) {
        return 0; // Base case: if the node is null, return 0
    }
    // Count the current node + count from left subtree + count from right subtree
    return 1 + countNodes(root->left) + countNodes(root->right);
}
```

```cpp
// Function to calculate the sum of all nodes in a binary tree
int sumOfNodes(Node* root) {
    if (root == nullptr) {
        return 0; // Base case: if the node is null, return 0
    }
    // Sum the current node's value + sum from left subtree + sum from right subtree
    return root->data + sumOfNodes(root->left) + sumOfNodes(root->right);
}
```

Copy code

```
std::string postfixToInfix(const std::string& expression) {
    std::stack<std::string> stack;
    std::istringstream tokens(expression); // Stream to read tokens from the expression
    std::string token;

    while (tokens >> token) { // Read each token
        if (isalnum(token[0])) { // If the token is an operand (alphanumeric)
            stack.push(token); // Push operand onto the stack
        } else { // The token is an operator
            std::string operand2 = stack.top(); // Pop the first operand
            stack.pop();
            std::string operand1 = stack.top(); // Pop the second operand
            stack.pop();

            // Create the new infix expression and push it onto the stack
            std::string newExpression = "(" + operand1 + " " + token + " " + operand2 + ")
            stack.push(newExpression);
        }
    }

    return stack.top(); // The final result will be on top of the stack
}
```